

Generative 4 & Sustaining 3

Building Intelligence-Producing AI Systems

Retrieval · Context · Memory · Feedback — Guardrails · Orchestration · Recovery

Practical Study Guide

1. Introduction

After reading this guide, you can draw the complete architecture of an intelligence-producing AI system, explain what each component does, and describe the sustaining elements that prevent production failures.

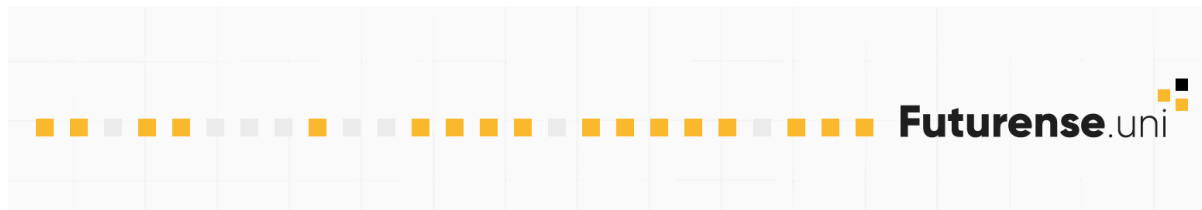
Sessions 1-3 established what to build and what to validate. This session answers: once you know what to build and which model to use, what exactly do you construct around that model?

The answer: seven components — four that produce intelligence, three that sustain it. Together they form the 80% of engineering work that the model cannot do itself.

Topics covered:

- Why models are stateless and what this forces you to build
- Satya Nadella: multi-model orchestration and harness engineering as IP
- Consistency revisited: accuracy vs Flip in domain decisions
- Why LLM eval data misleads and who does human feedback right
- Generative 4: Retrieval, Context, Memory, Feedback
- The MVC analogy mapping traditional to AI architecture
- Temperature as design configuration
- Sustaining 3: Guardrails, Orchestration, Recovery
- Air Canada case study: real failure from missing sustaining components

Key insight: After this guide: You can draw the full architecture, name each component's role, and diagnose the Air Canada failure in design terms.



2. Core Concepts

2.1 Why Models Are Stateless

A language model is stateless by default. Each API call is independent. The model processes text, returns text, retains nothing. ChatGPT feels stateful because the application re-sends the full conversation history with every call — statefulness is engineering, not model architecture.

This means everything that makes a model feel intelligent across a conversation — memory, domain facts, user context — must be constructed outside the model and passed with each call.

Key insight: The model is stateless. Intelligence feels stateful. You build everything that bridges that gap: context, retrieval, memory, feedback. These are architectural components, not model features.

2.2 Satya Nadella: Multi-Model, Harness Engineering, Context as IP

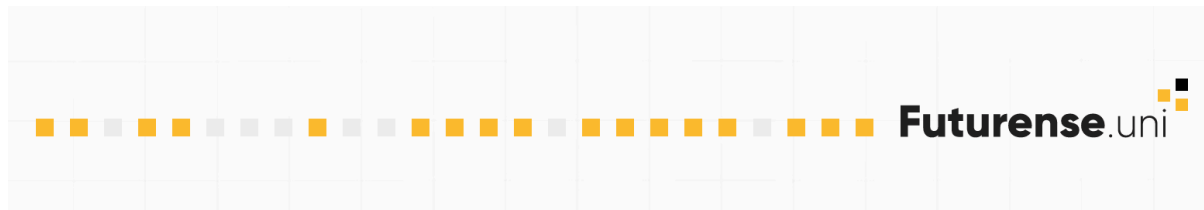
In a 2026 forum interview, Microsoft CEO Satya Nadella stated the AI architectural direction in three lines:

- The world is multi-model. No single model dominates. Different models for different tasks is the correct architecture.
- The IP of any application is how you orchestrate these models with context engineering and your data. Orchestration is the differentiation. Models are commodities.
- The reasoning traces from orchestration become your own capability. Perplexity's citation pipeline, Harvey's case law graph, an insurer's underwriting decision logic — these are moats competitors cannot replicate by switching models.

Translation: your system's value is not in which model you use. It is in how you combine retrieval, context, memory, and feedback for your domain. That combination — harness engineering — is your IP.

Harness Engineering = Retrieval + Context + Memory + Feedback + Orchestration

2.3 Consistency: Accuracy vs Flip



Accuracy: does the model give a correct answer? **Consistency:** does the model give the same answer repeatedly? These are different metrics. The difference is most dangerous in enterprise.

Four output patterns exist. **Lock:** identical output every call. **Stable:** minor wording variance, consistent core. **Learn:** improves with in-context feedback. **Flip:** opposite decisions on identical inputs. The Flip pattern appears most in domain-specific decision queries — the exact 10% of queries that represent 90% of enterprise risk.

Example: same underwriting application — approve on Monday, reject on Wednesday. Not a code bug. The model's probability for approve and reject is nearly equal at this boundary case. Sampling variance flips the result. If your eval only tests mainstream cases, you miss this entirely.

Pattern	What Happens	Enterprise Risk
Lock	Identical output every call	Low
Stable	Minor wording variance, consistent decision	Low for general; test for domain decisions
Learn	Improves with feedback	Manageable — design feedback loop
Flip	Opposite decisions on identical inputs	Critical — deploy without consistency test = random enterprise decisions

Run explicit Flip-rate tests on domain decision queries before deployment.

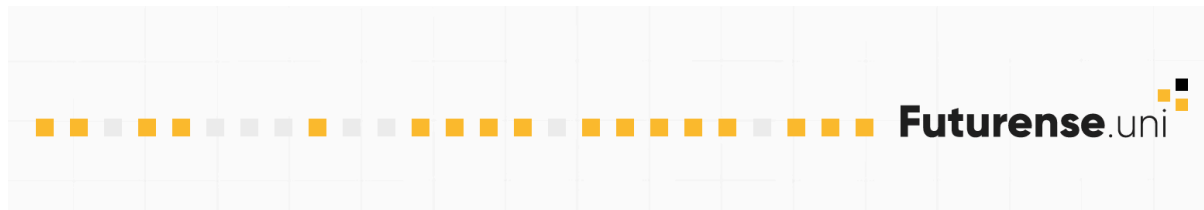
Watch out: A 90% accurate model can flip on 40% of its domain edge cases. Those 4% of all queries are the highest-stakes ones. Overall accuracy hides this. Domain flip testing reveals it.

2.4 Eval Data: Why LLM-Generated Data Misleads

When you ask an LLM to generate eval test cases, the generator uses the same training distribution as the model being tested. Both share the same blind spots. Your accuracy metrics look good. Your production system fails on cases neither system ever covered.

The correct solution: domain experts generating test cases from real historical decisions. Scale AI and Surge AI built businesses around this. Mercor went further — building a tool to identify which experts in a pool of 100,000 actually have genuine domain knowledge (typically 30-40%) before using their labeled data.

Watch out: Trust = f(Accuracy, Consistency). Accuracy measured on synthetic data is accuracy on biased tests. Real production trust will be lower.



3. Generative 4: The Four Intelligence-Producing Components

None of the Generative 4 live inside the model. All are your engineering responsibility.

3.1 Retrieval (RAG)

Retrieval solves the foundational problem: the model has general knowledge but no domain facts. An insurance model trained on 14 trillion tokens does not know your underwriting rules, product eligibility criteria, or regulatory constraints.

The SMB loan example: without retrieval, the model answers from generic credit knowledge. With retrieval, you fetch your SMB loan product specification (exact approval criteria, income requirements, liability limits), include it in the prompt, and the model reasons against your actual business rules.

Three properties distinguish good retrieval:

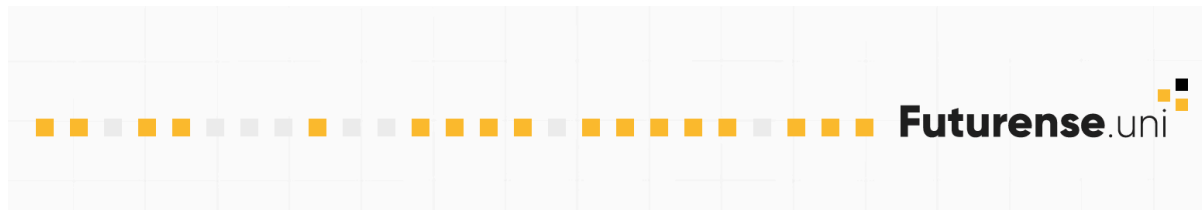
- **Relevance:** retrieve only sections relevant to this query (2-3 pages of a 50-page policy, not all 50).
- **Freshness:** retrieved content must reflect current policies. Set a content TTL and refresh proactively. Stale retrieval = confident wrong answers.
- **Precision:** dense semantic embedding retrieval outperforms keyword (BM25) matching for complex domain questions.

Key insight: Retrieval is domain grounding. Without it, the model answers from general knowledge. With it, from your business facts. The quality of retrieval is the ceiling of intelligence.

3.2 Context

Context solves the statefulness problem within a session. By query 4 of a 15-turn conversation, the model needs subject from query 1, constraints from query 2, conclusions from query 3 — without you resending the full growing history.

Context is the engineered solution: a maintained summary of the conversation's subject, relevant constraints, and extracted facts, sent with each query. Smaller than full history, more structured, always current.



Web engineering analogy: context in AI architecture is what session cookies are in traditional web. A stateless web server uses cookies to maintain user state across requests. A stateless language model uses context to maintain conversation state across calls.

Context is not retrieval. Retrieval provides the full domain document (30-page loan policy). Context stores the relevant summary for this conversation (the 3 rules that apply to this applicant). Retrieval = library. Context = extracted excerpt for this session.

3.3 Memory

Memory solves cross-session continuity. Context handles within-session. Memory handles across-session. When a user returns two weeks later, the system should know prior conclusions and not restart from zero.

Four memory types:

- **Short-term / per-turn: last 3-5 query/answer pairs in the current conversation.**
- **Episodic / within-session: full log of this session — what was asked, answered, agreed, rejected.**
- **Long-term / across-session: distilled conclusions from multiple previous sessions. Not raw logs — abstracted: this customer was approved for X, denied for Y; this edge case always fails at Z.**
- **Semantic / distilled: patterns from many episodes across all users. When 1000 applications show consistent patterns, those materialize as semantic memory for all future decisions.**

Cache analogy: short-term = L1 cache (fast, small, ephemeral). Long-term = database cache (slower, persistent). Semantic = query cache (patterns from repeated operations).

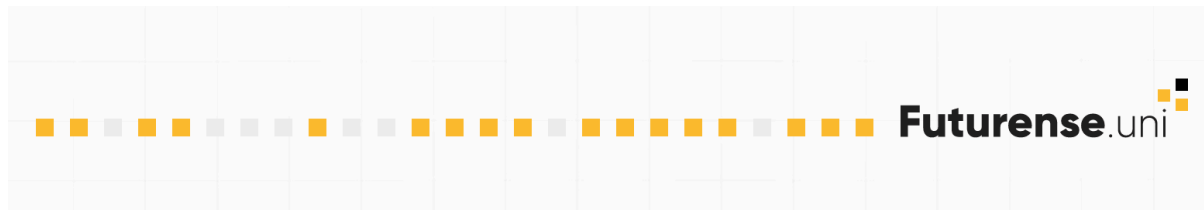
3.4 Prompt Assembly and Feedback

Prompt assembly is the final step: combining query + retrieved facts + context + memory + system instructions into one structured input. The model receives everything and reasons across it all.

Prompt ordering matters. Models weight recent context more heavily. Instructions at the beginning are followed more reliably. These are properties to measure in eval and design around.

Feedback is the learning mechanism. Without it, the system produces the same quality forever. Three types:

- **Human feedback: explicit corrections from domain experts. Highest quality, lowest volume. Used for fine-tuning and semantic memory.**
- **Implicit feedback: behavioral signals (user re-asked, edited output, escalated to human).**
- **Error feedback: failures caught by guardrails. High quality for specific failure modes.**



Key insight: Intelligence improves through feedback. A system without a feedback loop produces the same quality output forever. Design the feedback loop before deployment.

4. The MVC Analogy

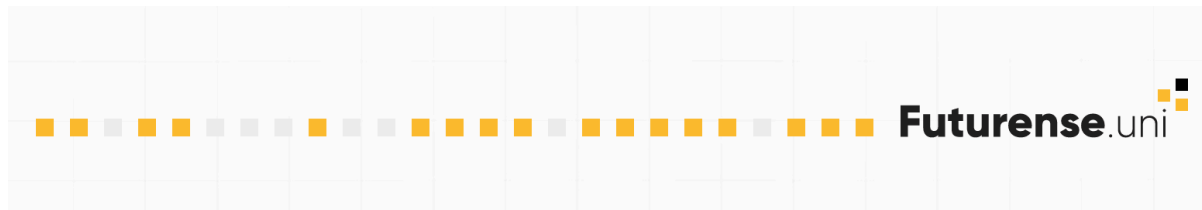
For engineers familiar with Model-View-Controller, the Generative 4 maps exactly to familiar concepts. The model is stateless, just like a stateless web server. The architecture solves the same problem: maintaining coherent state around a stateless engine.

Traditional MVC	Generative 4 Equivalent	Function
Model (data layer)	Retrieval (RAG)	Ground truth facts from permanent storage
Controller (logic layer)	Context + Prompt Assembly	Manages state, applies business rules, assembles model input
View (presentation)	Response + Memory Update	Produces user output; records conclusions
Session / Cookies	Memory (short + long term)	Maintains state across stateless requests
DTO / Business objects	Context extraction	Extracts only relevant fields for current operation

If you have built MVC applications, you have built most of these concepts. The difference: probabilistic outputs instead of deterministic ones.

The key insight: your system's value does not come from the LLM (commodity) any more than a web app's value comes from the HTTP server (commodity). It comes from the quality of your data layer, controller, and session management. That is where your IP lives.

5. Sustaining 3: Preventing Intelligence Failure



The Sustaining 3 prevent the intelligence you built from degrading, drifting, or causing harm. They are not optional features — they are the constraints that make production deployment safe.

5.1 Guardrails

Guardrails impose behavioral constraints to prevent intelligence failures from reaching users. Models have four documented enterprise failure modes without guardrails:

- **Drift:** the model generalizes exception cases into policy. The Air Canada chatbot extended a bereavement discount policy beyond its intended scope because its helpful-behavior pattern overrode the policy constraint.
- **Overconfidence:** models express high confidence on borderline questions. Enterprise requires calibrated confidence scores that match actual accuracy rates.
- **Policy enforcement failure:** correctly understanding a policy but misapplying it to edge cases the policy explicitly excludes.
- **Output validation failure:** syntactically correct output that is semantically invalid — loan amount exceeding the product maximum, logically impossible date ranges, recommendations that contradict compliance rules.

Two levels: pre-model (validate and sanitize input) and post-model (validate output against rule engines and compliance checkers before delivering to user).

Exception analogy: in traditional software you wrote exception blocks for system, business, and catastrophic failures. Guardrails are the same design pattern for probabilistic outputs. Catch the exception. Handle gracefully. Do not let model uncertainty reach the user as a wrong answer.

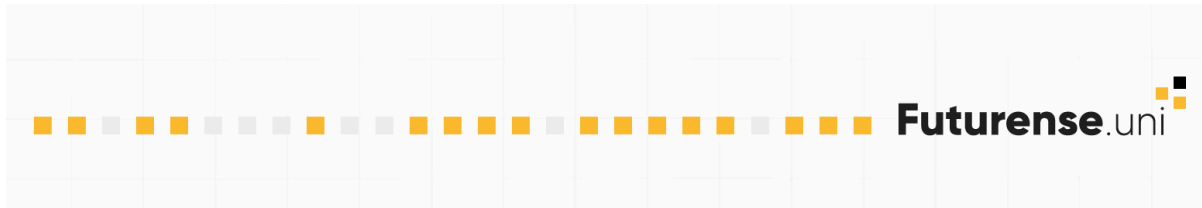
5.2 Orchestration

Orchestration coordinates all Generative 4 components. When a user sends a query, orchestration decides: which retrieval source, what context, which memory, which model, how to assemble the prompt, what to do with the response.

Orchestration is your responsibility. The model does not decide which source is relevant. You design that logic — encoding your domain understanding of how intelligence should flow. This is why Nadella says orchestration is your IP.

Four concerns require explicit design:

- **State management:** tracking conversation state, accessed sources, accumulated context.
- **Routing logic:** which model for which task (small for FAQ, large for policy decisions). This is multi-model architecture in practice.
- **Observability:** logging every decision. Without logs, you cannot diagnose failures. Every component must be instrumented.



- **Composition:** chaining multiple intelligence steps. Some tasks require extract → analyze → verify → respond. Designing these chains is orchestration.

5.3 Recovery

Recovery designs explicit paths for what happens when failures occur despite guardrails and orchestration. Three tiers:

- **Soft failures:** model is uncertain but not wrong. Deliver with confidence disclosure or offer to escalate. System continues.
- **Hard failures:** output violates a guardrail. Withhold output. Return safe fallback with explanation. Log for review.
- **Catastrophic failures:** system or dependency unavailable. Fail to a manual process — never to silence or a wrong answer.

Key insight: Guardrails + Orchestration + Recovery form the prevent-detect-recover loop. Design all three before deployment.

6. Case Study: Air Canada

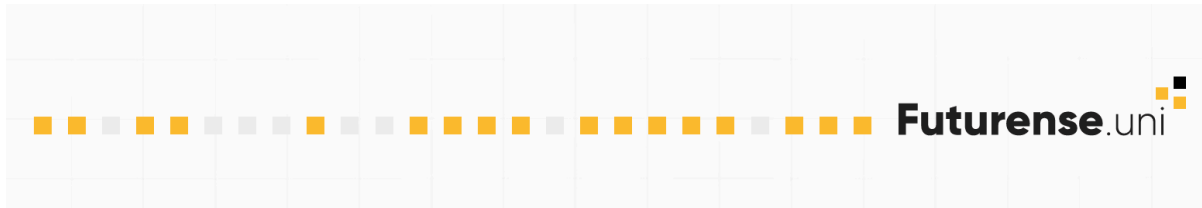
In 2024, Air Canada's AI chatbot told a customer that bereavement fares could be claimed retroactively. The actual policy required advance booking. The customer sued. A Canadian tribunal ruled against Air Canada, holding the company responsible for its AI's information. Air Canada paid.

This case illustrates all three Sustaining 3 failures simultaneously.

Guardrail failure: the model drifted, extending the bereavement policy beyond its scope because its helpful-behavior pattern overrode the policy constraint. No output validation guardrail caught the violation before delivery.

Orchestration failure: no routing logic detected that this query involved a specific named policy requiring retrieval of the exact policy text. The model answered from general knowledge instead of the retrieved policy.

Recovery failure: no recovery path existed. The system delivered the wrong answer directly to the customer with no confidence disclosure, no escalation offer, no audit trail.



Correct architecture: query about bereavement fares → retrieve exact policy text → assemble prompt with query + policy → post-model compliance check → if violation detected, return safe fallback with escalation option → log the event.

Watch out: Enterprise AI failures that reach customers are not recoverable with 'the AI was wrong.' You are responsible for what your AI tells your customers.

7. Code Walkthroughs

7.1 RAG Pipeline

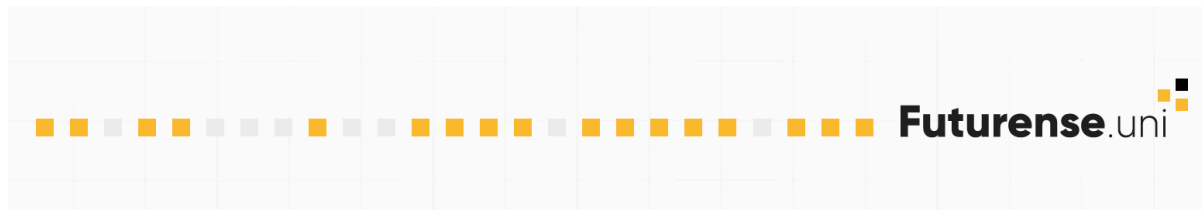
Step 1: Domain knowledge store and retrieval

```
import anthropic
client = anthropic.Anthropic()

KNOWLEDGE_BASE = {
    'smb_loan': '''
        SMB Loan Eligibility - Luna Pro:
        Revenue: minimum INR 50 lakh per year
        Business age: minimum 2 years registered
        Credit score: minimum 680 CIBIL
        No defaults in last 24 months
        Maximum loan: INR 2 crore
        Exception: thresholds waivable with senior approval
    '''
}

def retrieve(query):
    for key, doc in KNOWLEDGE_BASE.items():
        if any(kw in query.lower() for kw in ['loan', 'smb', 'credit', 'approve']):
            return doc
    return ""

def rag_query(user_query):
    facts = retrieve(user_query)
```



```
prompt = f'Policy:\n{facts}\n\nQuery: {user_query}\nAnswer from policy only.'
msg = client.messages.create(model='claude-haiku-4-5-20251001',max_tokens=200,
    messages=[{'role':'user','content':prompt}])
return msg.content[0].text
```

```
print(rag_query('3-year-old business, CIBIL 710. Approve SMB loan?'))
```

Output:

CIBIL 710 exceeds the 680 minimum. Business age 3 years exceeds the 2-year minimum. Eligible subject to revenue verification (INR 50L+/year) and 24-month default check.

7.2 Context + Memory Layer

Step 2: Add conversation context and short-term memory

```
from dataclasses import dataclass, field
from typing import List, Dict

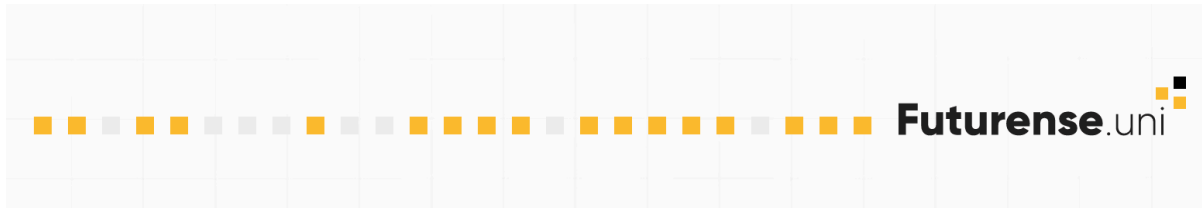
@dataclass
class Session:
    user_id: str
    context: str = ""
    short_memory: List[Dict] = field(default_factory=list)
    long_memory: str = ""

sessions: Dict[str, Session] = {}

def get_session(uid):
    if uid not in sessions: sessions[uid] = Session(user_id=uid)
    return sessions[uid]

def update_session(s, user_msg, assistant_msg):
    s.short_memory.append({'u': user_msg, 'a': assistant_msg})
    s.short_memory = s.short_memory[-3:]
    s.context = f'Topic: {user_msg[:80]}. Last: {assistant_msg[:80]}'

def conversational_rag(uid, query):
    s = get_session(uid)
    facts = retrieve(query)
    recent = '\n'.join([f'U: {t["u"]}\nA: {t["a"]}' for t in s.short_memory])
    prompt = (f'Context: {s.context}\nPrior conclusions: {s.long_memory}\n'
```



```
f'Recent turns:\n{recent}\nDomain facts:\n{facts}\nQuery: {query}')
msg = client.messages.create(model='claude-haiku-4-5-20251001',max_tokens=200,
messages=[{'role':'user','content':prompt}])
resp = msg.content[0].text
update_session(s, query, resp)
return resp

uid = 'user_001'
r1 = conversational_rag(uid, 'What are SMB loan criteria?')
r2 = conversational_rag(uid, 'Business is 3 years old, CIBIL 650. Eligible?')
print('Turn 1:', r1[:100])
print('Turn 2:', r2[:100])
```

Output:
Turn 1: Criteria: revenue INR 50L+, 2+ years, CIBIL 680+, no defaults in 24 months. Turn 2: Based on criteria discussed: CIBIL 650 falls below 680 minimum. Not eligible for automatic approval.

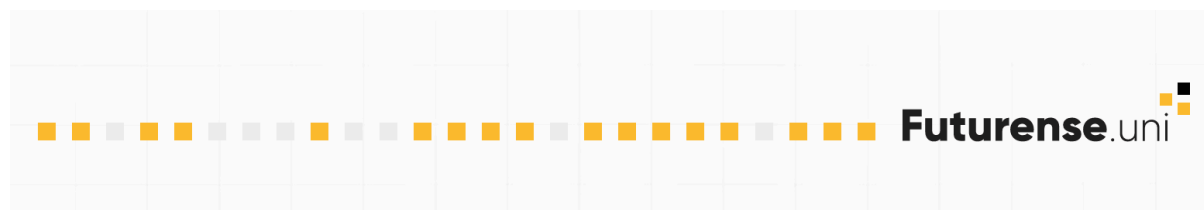
Turn 2 correctly references Turn 1 without the user repeating criteria. This is the context layer at work, not the model's memory.

7.3 Post-Model Guardrail

Step 3: Add output validation before delivering to user

```
def guardrail_check(response, domain_facts):
    prompt = (f'Policy:\n{domain_facts}\n\nResponse:\n{response}\n\n'
              f'Does this response contradict or misapply the policy?\n'
              f'Answer: SAFE or VIOLATION: [reason]')
    msg = client.messages.create(model='claude-haiku-4-5-20251001',max_tokens=60,
    messages=[{'role':'user','content':prompt}])
    r = msg.content[0].text.strip()
    if r.startswith('SAFE'): return True, ""
    return False, r.replace('VIOLATION:', '').strip()

def guarded_rag(query):
    facts = retrieve(query)
    response = rag_query(query)
    is_safe, reason = guardrail_check(response, facts)
    if is_safe: return response
    return f'Please consult a loan officer. (Note: {reason})'
```



```
print(guarded_rag('CIBIL 650 but great revenue. Can I still qualify?'))
```

Output:

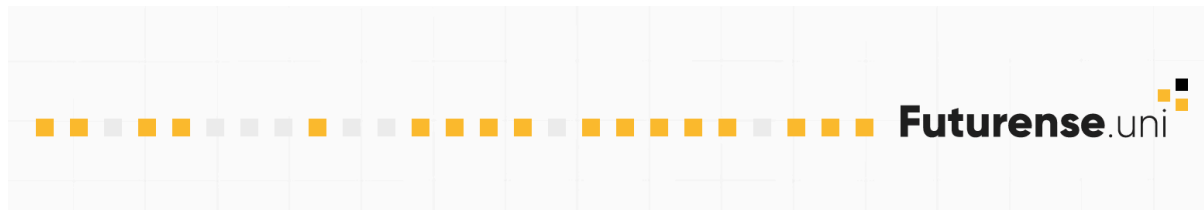
Please consult a loan officer. (Note: response suggested exception approval without mentioning senior approval requirement)

Key insight: This guardrail prevents an answer that would tell the user they might qualify by exception, without disclosing that exceptions require senior approval. In the Air Canada case, this component did not exist.

8. Design Decision Framework

Component	Key Design Questions	Common Mistake
Retrieval	How fresh? Semantic or keyword? Which sources are authoritative?	Retrieving entire documents; using stale sources
Context	Generic rules or user-specific? When to summarize? What triggers reset?	Growing without summarization until context window overflows
Memory	Short-term: how many turns? Long-term: what to abstract? When to update?	No memory design; storing everything (expensive and noisy)
Feedback	Who provides it? How does it update memory? Signal quality?	No feedback loop; relying on synthetic eval data
Guardrails	Pre- or post-model? What are hard limits? How to handle violations?	Only happy-path testing; no output validation
Orchestration	Which model per task? How to route? What to log?	One model for all tasks; no logging
Recovery	Failure tiers? Safe fallback? How to alert?	No recovery design until first production incident

Answer every question before beginning implementation.



9. Common Mistakes

Mistake 1: Building the 20% before the 80%

The 20% is the model. The 80% is everything else. Most engineers build the model integration first because it produces visible demo results. A demo without the 80% fails in production. Build the 80% first.

Mistake 2: Treating the model as stateful

The model is stateless. Every coherence you see in ChatGPT is engineered by the application layer. Every coherence in your system must be engineered by you.

Mistake 3: No retrieval freshness policy

Retrieval sources go stale. A loan policy from 6 months ago may no longer be accurate. Set a content TTL and refresh mechanism before deployment.

Mistake 4: Storing raw conversation logs as memory

Raw logs are expensive and the model weights recent turns most heavily, losing earlier conclusions. Design summarization that distills each session to key conclusions.

Mistake 5: Shipping without guardrails

Guardrails feel like overhead when the demo works. They prevent the Air Canada outcome. Design them before you write the first model call.

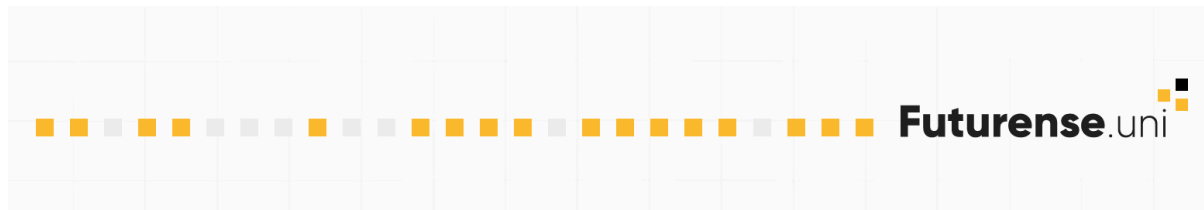
Mistake 6: No observability

Orchestration without logging is a black box. When the system fails — and it will — you will not know which component failed without logs. Instrument everything.

10. Interview Preparation

Q1: What is the difference between retrieval and context?

Answer: Retrieval fetches domain facts from a knowledge store — the actual business rules and policy documents the model needs to answer domain-specific questions. Context is the engineered conversation state — a compressed summary of the session's subject, relevant extracted facts, and



conclusions so far. Retrieval is the library. Context is the relevant chapter extracted for this session. Different freshness requirements, different storage, different update mechanisms.

Q2: Why is the model stateless, and what does this require?

Answer: A language model is stateless because each API call is independent. All conversation continuity, domain grounding, and user context must be constructed outside the model and passed with each call. In practice: maintain context (conversation summary), retrieval (domain facts), and memory (cross-session conclusions). Assemble all three into a structured prompt before each model call. The model appears stateful because you engineered statefulness around a stateless engine.

Q3: What happened in Air Canada and how would you have prevented it?

Answer: Air Canada's chatbot incorrectly told a customer that bereavement fares could be claimed retroactively. Air Canada was held legally liable. Three design failures: no retrieval (answered from general knowledge, not the actual policy), no output guardrail (no compliance check caught the policy violation), and no recovery (no escalation path for policy queries). Prevention: RAG to retrieve the exact policy for any named-fare query, post-model compliance guardrail validating against retrieved policy, recovery path escalating policy queries to a human agent.

Q4: Why would you use low temperature for enterprise underwriting?

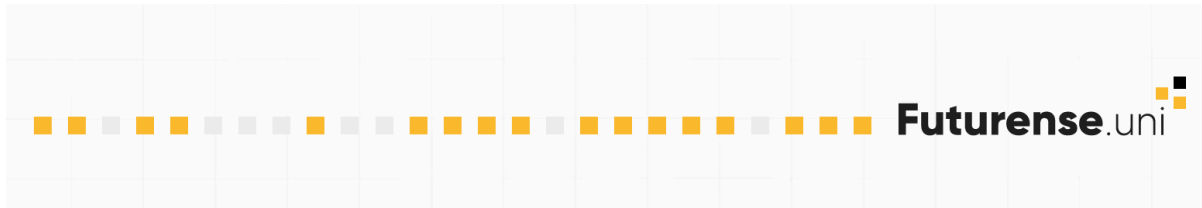
Answer: Temperature controls output randomness. High temperature increases hallucination rate. Low temperature makes outputs more deterministic. For enterprise underwriting, the same application must produce the same decision on repeated calls (consistency requirement) and wrong answers create legal liability. Low temperature minimizes the Flip pattern — opposite decisions for the same borderline application on different calls. Measure flip rate at temperature 0, 0.2, and 0.5 for domain decision queries, confirm 0 or 0.2 meets the enterprise consistency threshold, and document this in the system specification.

11. Practice Problems

11.1 MCQs

Q1. A user on their fourth loan query gets an answer that ignores constraints established in query one. What component is missing?

- A) Retrieval — did not fetch the loan policy.
- B) Context — did not maintain conversation state across turns.
- C) Guardrail — did not validate against policy.



D) Memory — did not retain prior session conclusions.

Answer: B. Within-session continuity is context's responsibility. Context summarizes what was established in previous turns and passes it with each query. Missing context means the model treats each query as independent.

Q2. A user returns two weeks later. The system treats them as new. What is missing?

A) Retrieval.

B) Context.

C) Long-term memory — no mechanism preserves conclusions between sessions.

D) Guardrail.

Answer: C. Cross-session continuity requires long-term memory. Context handles within-session. Memory handles across-session.

Q3. The Air Canada chatbot told a customer bereavement fares were retroactive. The actual policy says they are not. Which Sustaining 3 component would have prevented this?

A) Orchestration — routing to a different model.

B) Recovery — escalating to a human agent.

C) Guardrail (output validation) — post-model check catching the policy violation before delivery.

D) Retrieval — not accessing the policy.

Answer: C. A post-model output guardrail validates the response against the retrieved policy before delivery. The response violated the explicit policy. This guardrail would have intercepted it.

Q4. Same underwriting application: approve Monday, reject Wednesday. Most likely cause?

A) Retrieval source updated between calls.

B) Model Flip pattern on domain edge case.

C) Context window exceeded.

D) Temperature too low.

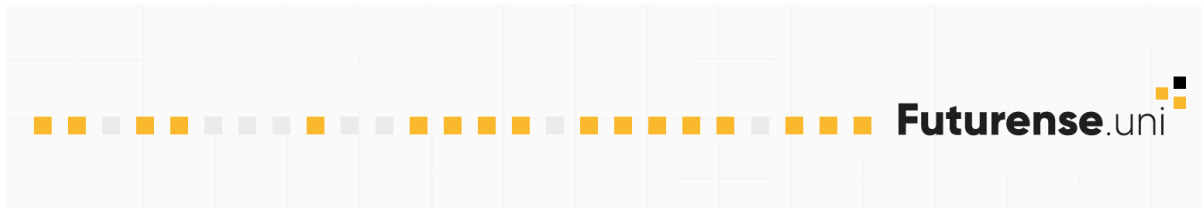
Answer: B. Flip occurs when probability for opposite decisions is nearly equal at a boundary case. Sampling variance produces opposite results on different calls.

Q5. Satya Nadella says the IP of any application is how you orchestrate models with context engineering and your data. Which architectural choice reflects this?

A) Choosing the most powerful model for all tasks.

B) Patent-protecting your prompt.

C) Designing retrieval, context, memory, and orchestration that make the model intelligent for your domain.



D) Fine-tuning the model on proprietary data.

Answer: C. The model is a commodity. The differentiation is in the harness: retrieval sources, context engineering, memory architecture, and orchestration. That combination is your IP.

Q6. Which best describes the relationship between retrieval and context?

A) Retrieval is short-term; context is long-term.

B) Retrieval provides raw domain facts; context compresses them into relevant conversation state.

C) They are the same component with different names.

D) Context provides facts; retrieval manages conversation state.

Answer: B. Retrieval fetches the full domain document. Context extracts and compresses the relevant rules for the current conversation. Library access vs working memory.

Q7. Why is LLM-generated eval data problematic?

A) Insufficient volume.

B) It mirrors the same training distribution as the tested model, masking shared blind spots.

C) Too expensive.

D) Cannot generate domain questions.

Answer: B. Circular validation: eval generator and tested model share the same blind spots. The production failures that matter are exactly the cases neither system covers.

Q8. For a refund-processing customer service bot, which temperature setting is most appropriate?

A) High (0.9) for natural varied responses.

B) Low (0 to 0.2) for consistent policy application and minimum Flip risk.

C) Medium (0.5) as a balanced compromise.

D) Temperature is irrelevant.

Answer: B. Refund decisions carry financial liability. The same request must receive the same decision on repeated calls. Low temperature maximizes consistency and minimizes the Flip pattern.

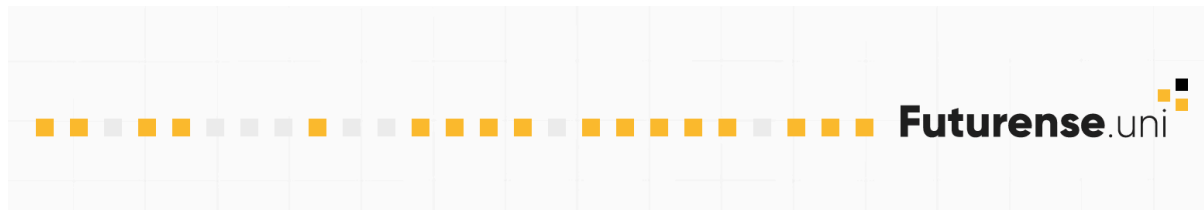
Q9. An orchestration layer routes all queries to a large expensive model. Primary problem?

A) Large models cannot handle FAQ queries.

B) Over-engineering simple tasks wastes budget that should be reserved for high-complexity, high-risk decisions.

C) Large models are incompatible with RAG.

D) This violates Satya Nadella's principle.



Answer: B. Route by task type and risk. Small models for FAQ (low stakes), large models for policy decisions (high stakes). Using large model for everything is expensive and unnecessary.

Q10. Code: ``if conf < 0.65: return 'Please consult a loan officer.'`` What pattern is this?

- A) Retrieval augmentation.
- B) Memory update.
- C) Recovery — safe fallback when confidence falls below deployment threshold.
- D) Context reset.

Answer: C. Recovery pattern: when confidence is below the acceptable threshold, the system falls back to a safe message directing the user to a human expert. Graceful degradation under uncertainty.

11.2 Assignments

Assignment 1: Design the Generative 4 for Your Domain

Apply all four components to your extraordinary problem from Sessions 2 and 3:

- **Retrieval:** name 3 specific sources, explain their freshness requirement.
- **Context:** write a context template showing the fields it would contain.
- **Memory:** design short-term, long-term, and semantic memory for your system.
- **Feedback:** name one explicit and one implicit signal that would update your memory.

Submit: one table with all four components. One paragraph on the hardest design decision.

Assignment 2: RAG with Guardrails

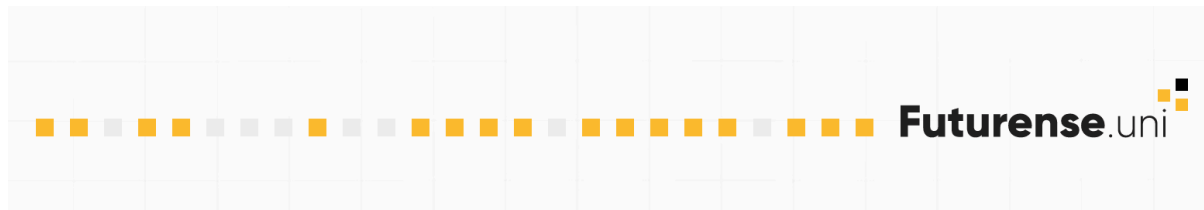
Build a working RAG + guardrail system for any domain you know:

- Knowledge base with at least 3 policy documents or rules.
- Retrieval function returning relevant documents.
- Model call combining retrieved documents with user query.
- Post-model guardrail checking response against policy.
- Recovery function returning safe fallback when guardrail fires.

Test with 3 queries: one passes, one fires the guardrail, one edge case. Show all three outputs.

Assignment 3: AI Failure Case Analysis

Choose any publicly reported AI failure (Air Canada, Amazon Rekognition, Microsoft Tay, Zillow pricing algorithm). Write 500 words analyzing which Sustaining 3 components failed, what design decision was missing, and how you would have designed it differently.



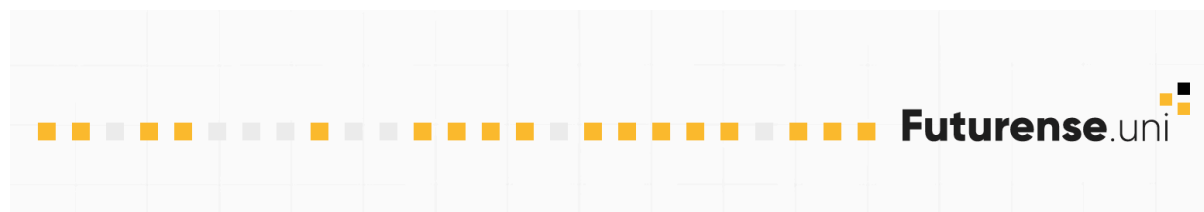
12. Further Learning

- **Satya Nadella at World Economic Forum 2026: the 3-minute multi-model segment on orchestration as IP.** Source for Section 2.2. Watch three times.
- **LangChain Memory documentation: ConversationBufferMemory (short-term), ConversationSummaryMemory (context), VectorStoreRetrieverMemory (semantic).** Technical implementation of Section 3.3.
- **LlamaIndex RAG best practices: retrieval chunking, re-ranking, hybrid search, context compression.**
- **Air Canada chatbot bereavement fare ruling: the full tribunal decision is publicly available.** Complete case study for Section 5.
- **Temperature, top-k, top-p sampling: the mathematical basis for Section 2.3 and temperature configuration.**
- **Anthropic Model Context Protocol (MCP): the emerging industry standard for model-tool orchestration.**
- **Scale AI and Mercor: how human-domain feedback is collected at scale.** Context for Section 2.4.

13. Cheatsheet

Generative 4

Component	Analogy	What It Does	Key Design Choice
Retrieval (RAG)	Library access	Fetches domain facts from knowledge store	Freshness TTL, semantic vs keyword, chunk size
Context	Session cookies / DTO	Maintains compressed conversation state	Generic vs specific; when to summarize



Memory	Cache (L1/L2/query)	Preserves conclusions across turns and sessions	What to abstract, retention, update cadence
Prompt Assembly + Feedback	Controller + learning loop	Combines everything; learns from outcomes	Signal quality, who provides feedback

The model does none of this. You do all of it.

Sustaining 3

Component	What It Does	Air Canada Failure
Guardrails	Prevents policy violations from reaching users	No output validation; wrong answer delivered directly
Orchestration	Coordinates all components; routes by task; logs everything	No retrieval routing; no logging
Recovery	Safe fallback when failure is detected	No escalation path; no human handoff option

All three must be designed before deployment.

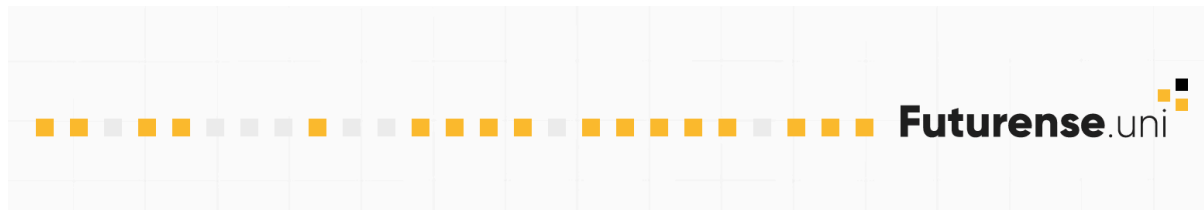
Memory Types

Type	Scope	What It Stores
Short-term	Per-turn	Last 3-5 query/answer pairs
Episodic	Within-session	Full current session log
Long-term	Across-sessions	Distilled conclusions from previous sessions
Semantic	Domain-wide	Learned patterns from many user sessions

Design all four. Short-term alone loses conclusions between sessions.

Key Formulas

Harness Engineering = Retrieval + Context + Memory + Feedback + Orchestration



Trust = $f(\text{Accuracy, Consistency})$ — measure both, deploy only when both meet enterprise threshold

Intelligence = 20% model + 80% architecture — you build the vehicle around the engine

Recovery tier 1 (soft): answer with disclosure | tier 2 (hard): safe fallback | tier 3: fail to manual

Golden Rules

- **The model is stateless. Engineer statefulness through context, memory, and retrieval.**
- **Retrieval is domain grounding. Without it, the model answers from general knowledge.**
- **Context is within-session memory. Memory is across-session memory. They are different components.**
- **Every enterprise system needs guardrails. No exceptions.**
- **Orchestration is your IP. Design it deliberately. Log every decision.**
- **Recovery is designed before deployment, not after the first incident.**
- **Domain-expert eval data beats LLM-generated eval data, always.**
- **Flip rate on domain decisions is more dangerous than overall accuracy. Test for it explicitly.**
- **Temperature for enterprise decisions: 0 to 0.2. For creative tasks: higher.**

Mental model first. Implementation second. Tools